1. **Опишите, как устроена обработка HTTP-запроса в net/http: что такое http.Server, Handler, ServeMux, и как запрос проходит через middleware до конкретного handler'a.**
   - **http.Server**: Структура, конфигурирующая параметры сервера (таймауты, адрес, обработчик). Метод `ListenAndServe` запускает бесконечный цикл: `Accept` соединения -> создание горутины -> чтение запроса.
   - **Handler**: Интерфейс с одним методом `ServeHTTP(ResponseWriter, *Request)`. Любой объект, реализующий его, может обрабатывать запросы.
   - **ServeMux**: Стандартный маршрутизатор (роутер). Он выбирает наиболее подходящий зарегистрированный паттерн для URL запроса.
   - **Цепочка**: Запрос попадает в `Server` -> `ServeMux` (как главный Handler) -> Middleware 1 -> Middleware 2 -> Конечный Handler.
2. **Как бы вы спроектировали единый формат ошибок для HTTP API (коды, сообщения, детали, request_id)? Какие поля вы бы включили и почему?**
   Структура, которая содержит следующие поля:
   - `code` - код ошибки
   - `msg` - читаемое сообщение для разработчкиков
   - `request_id` - айдишник для поиска, например, в логах
   - Можно ещё дополнительно какое-нибудь поле `details`, где будет подробно что-то описываться, например, все ошибки валидации полей
3. **Расскажите, как вы реализуете ручную валидацию входящих JSON-запросов в Go без сторонних библиотек: где она должна жить, как тестируется, как возвращать ошибки по полям.**
   - **Где**: В слое Handler (сразу после `json.Decode`) или в методе `Validate()` самой структуры запроса.
   - **Как**: Простые `if` проверки.
   - **Возврат**: Карта `map[string]string`, где ключ — имя поля, значение — текст ошибки.
   - **Тестирование**: Unit-тесты на функцию `Validate` с передачей валидных и невалидных структур.
4. **Объясните разницу между PUT и PATCH в REST API. Как вы бы реализовали PATCH так, чтобы отличать "поле отсутствует" от "стереть значение в этом поле"?**
   - **PUT**: Полная замена ресурса. Если поле не прислано, оно должно затереться дефолтным значением в БД.
   - **PATCH**: Частичное обновление.
   - **Проблема "nil vs zero"**: Чтобы отличить "не прислали поле" от "прислали null/пусто", используют указатели в структуре:

```go
type UpdateUser struct {
    Name *string `json:"name"` // nil - не меняем, *"" - стереть имя
}
```

5. **Как вы организуете маршрутизацию и извлечение path-параметров в Go 1.22+ с http.ServeMux? Какие есть ограничения и как вы их обходите?**
   - **Новое**: Теперь поддерживаются методы (`GET /users`) и переменные пути (`/users/{id}`).
   - **Извлечение**: `id := r.PathValue("id")` .
   - **Ограничения**: Нет поддержки регулярных выражений (regex) в путях, то есть все параметры являются строками

     > Например, нельзя указать, что в `/users/{id}` id должно быть именно числом

   - **Обход**: Проверять вручную внутри хэндлеров (либо использовать сторонние роутеры, например, chi).
6. **Опишите подход к graceful shutdown для HTTP-сервера: какие сигналы обрабатываются, что делает Shutdown, какие таймауты важны и почему.**
   1. Перехватываем сигналы ОС (`SIGINT` , `SIGTERM`).
   2. При получении сигнала вызываем `server.Shutdown(ctx)` .
   3. `Shutdown` перестает принимать новые запросы и ждет завершения активных
   4. **Таймауты**: Обязательно передаем контекст с таймаутом, чтобы сервер не висел вечно, если запрос "залип".
7. **Что такое middleware в net/http и как вы строите цепочку middleware? Приведите пример порядка middleware (request_id, logging, auth, recovery) и объясните, почему порядок важен.**
   Эта функция, которая обрабатывает запрос до/после основного handler *(например, проверка авторизации, логирование)*, принимающая `http.Handler` и возвращающая `http.Handler` .

Middleware	Зачем здесь?	Причина
Recovery	1-й уровень	Должен стоять первым, чтобы «поймать» `panic` в любом из последующих слоев (включая логирование или авторизацию). Если паника случится в `Logging` , а `Recovery` стоит внутри него, приложение упадет.
RequestID	2-й уровень	Генерирует уникальный ID для запроса. Нам нужно, чтобы этот ID уже существовал до того, как сработает логер.
Logging	3-й уровень	Записывает данные о запросе. Поскольку `RequestID` уже сработал, логер сможет записать этот ID, что критически важно для отладки (tracing).

Middleware	Зачем здесь?	Причина
Auth	4-й уровень	Проверяет права. Мы ставим её ближе к концу, чтобы не тратить ресурсы на проверку токена, если запрос «кривой» или вызвал ошибку на ранних этапах. Кроме того, логер выше успеет зафиксировать попытку входа.

8. **Объясните, что такое ETag/If-Match и как это помогает реализовать оптимистическую конкуренцию. Как бы вы реализовали это для ресурса в памяти и в базе данных?**
   ETag (Entity Tag) — это уникальный идентификатор конкретной версии ресурса. Это может быть хеш контента, версия (integer) или метка времени.
   - **GET запрос**: Сервер отдает ресурс и заголовок `ETag: "v1"`.
   - **Клиент**: Хранит этот ETag. Когда он хочет обновить ресурс, он отправляет заголовок `If-Match: "v1"`.
   - **Сервер**: Сравнивает `If-Match` из запроса с текущим ETag ресурса.
     - Если совпадают — обновляем данные и генерируем новый `ETag: "v2"`.
     - Если **не** совпадают — возвращаем **412 Precondition Failed**. Клиент должен заново загрузить данные и применить изменения
       Реализация: В БД добавляем колонку `version` (int) или `updated_at`. `UPDATE ... WHERE id = 1 AND version = old_version`.
9. **Расскажите, как работает database/sql: что такое *sql.DB (пул), как он управляет соединениями, и почему его создают один раз на приложение.**
   `*sql.DB` — пул соединений. При выполнении запроса пул автоматически выбирает свободное соединение из списка «спящих» или открывает новое. После завершения запроса соединение не разрывается, а возвращается в пул для повторного использования другими горутинами. Это избавляет от необходимости тратить ресурсы на дорогостоящие TCP-рукопожатия и авторизацию при каждом обращении к базе.

   Если создавать новый `*sql.DB` на каждый запрос, приложение быстро исчерпает свободные порты ОС и перегрузит базу данных процессом постоянного входа и выхода.
10. **Какие настройки пула соединений (SetMaxOpenConns, SetMaxIdleConns, SetConnMaxIdleTime, SetConnMaxLifetime) вы бы выбрали для микросервиса и как бы обосновали эти значения?**
    - **SetMaxOpenConns**: Лимит всех соединений. *Микросервис:* 20–50 (зависит от нагрузки и лимита самой БД). Нужно, чтобы не «положить» базу.
    - **SetMaxIdleConns**: Сколько «горячих» коннектов держать в запасе. *Обоснование:* Лучше делать равным или чуть меньше MaxOpen, чтобы не тратить время на повторное TCP-рукопожатие.

- **SetConnMaxLifetime**: Максимальный возраст коннекта. *Зачем:* Чтобы корректно закрывать старые соединения, которые могут «протухнуть» на уровне сетевого оборудования/балансировщика.
- **SetConnMaxIdleTime**: Время жизни «спящего» коннекта. *Зачем:* Освобождать ресурсы базы в периоды затишья.

11. **В каких ситуациях возникает sql.ErrNoRows? Как вы маппите его на доменные ошибки и на HTTP/gRPC статусы?**
    - Возникает при `db.QueryRow`, если запрос не вернул ни одной строки.
    - **Маппинг**:
      1. В Repository: проверяем на `sql.ErrNoRows` и возвращаем доменную ошибку `ErrNotFound`.
      2. В Handler: маппим `ErrNotFound` -> **404 Not Found** (HTTP) или **StatusNotFound** (gRPC).
12. **Опишите, как вы проектируете repository layer: зачем он нужен, где проходит граница между handler/service/repository, и как это помогает тестированию.**
    - **Зачем**: Отделяет логику хранения от бизнес-логики.
    - **Границы**:
      - *Handler*: Парсинг входящих запросов.
      - *Service*: Бизнес-логика.
      - *Repository*: Работа с БД (SQL-запросы).
    - **Тестирование**: Можно создать интерфейс репозитория и в тестах сервиса подсунуть мок (mock), не поднимая реальную базу.
13. Как вы реализуете транзакции в Go (BeginTx, Commit, Rollback) и какие типичные ошибки допускают при работе с транзакциями? 4

```go
tx, err := db.BeginTx(ctx, nil) // Начало транзакции
defer tx.Rollback() // Обязательно, если завершилось без комита
...

err = tx.Commit() // Сохранение всех изменений
```

```
Ошибки:
    1. Использование объекта `db` вместо `tx` внутри транзакции.
    2. Забытый `Rollback` при ошибке (утечка соединений).
    3. Слишком длинные транзакции (блокировки таблиц).
    4. tx.Commit тоже может вернуть ошибку, не стоит забывать обрабатывать
```

14. **Объясните, что значит "добавить audit/event запись в той же транзакции". Какие гарантии это даёт и какие проблемы предотвращает?**
    - **Суть**: Запись о действии (например, «списаны деньги») и сам факт изменения баланса делаются одним `COMMIT`.
    - **Гарантии**: **Атомарность**. Либо сохранятся оба действия, либо ни одного.

- **Проблемы**: Предотвращает рассинхрон — когда деньги списались, а лог в базу не попал (или наоборот).

15. **Опишите JWT (HS256) на практическом уровне: какие claims вы используете, как проверяете подпись и срок действия, и как организуете middleware для аутентификации.**
    JWT состоит из трех частей, разделенных точками: `Header.Payload.Signature`.

    1. **Header**: Содержит тип токена (`JWT`) и алгоритм подписи (в нашем случае **HS256** — HMAC с использованием SHA-256).
    2. **Payload (Claims)**: Полезная нагрузка. Те самые утверждения (claims), которые мы передаем:
       - **Standard claims**: `sub` (ID пользователя), `exp` (время истечения), `iat` (когда выпущен).
       - **Custom claims**: `role` (admin/user), `email` , `org_id` .
    3. **Signature**: Результат хеширования Header + Payload с использованием вашего **Secret Key**. Она гарантирует, что данные в токене не были изменены.

    Middleware:
    1. Извлекает строку из заголовка `Authorization` .
    2. Парсит JWT и проверяет подпись.
       - Разбивает строку на части.
       - Берет Header и Payload из присланного токена и заново вычисляет подпись со своим секретным ключом.
       - Сравнивает вычисленную подпись с той, что прислал клиент. Если они не совпадают — токен подделан.
       - Проверяет поле `exp` . Если текущее время больше, чем в токене — токен протух.
    3. Если всё ок — извлекает `user_id` из Claims и кладет его в **context.Context** запроса.
    4. Если токен невалиден или протух — сразу возвращает **401 Unauthorized**, не пропуская запрос дальше к хендлеру.

16. **Как вы проектируете авторизацию поверх JWT: роли, claims, ownership (владелец ресурса). Где лучше размещать проверки — в handler'ах или в сервисном слое — и почему?**
    - **Ownership**: Проверка «владеет ли юзер этим ресурсом». Делается в **Service/Repository**, так как нужно лезть в БД и сравнивать `resource.user_id` с `token.user_id` .
    - **Где проверки**: Роли (admin/user) можно в middleware. Сложные правила (ownership) — только в слое логики.

17. **Опишите "refresh-like" подход для JWT: как вы бы сделали короткоживущий access token и механизм перевыпуска. Какие компромиссы по безопасности**

возникают?

Refresh-подход подразумевает два токена:

- access token (короткоживущий, например 15 минут)
- refresh token (долгоживущий, например 30 дней, хранится в БД или redis, либо как самодостаточный JWT с особыми ограничениями)

  Механизм:
- клиент использует access токен для API;
- при его истечении отправляет refresh token на специальный эндпоинт /refresh.
- Сервер проверяет refresh token (валидность, не отозван ли), после чего выдаёт новую пару токенов (или только новый access).
- Refresh token обычно хранится в httpOnly cookie или защищённом хранилище, а access — в памяти.

  Компромиссы по безопасности:
- если refresh токен скомпрометирован, злоумышленник может получать новые access токены до его отзыва.
- Сокращение времени жизни access токена уменьшает окно ущерба при краже, но увеличивает нагрузку на эндпоинт обновления.
- Отзыв refresh токенов требует stateful хранилища (БД/кэш), что нарушает безгражданность JWT.

18. **Как вы реализуете ротацию ключей для HS256 (current + previous secret)? Как это влияет на валидацию токенов и на процесс деплоя?**
    - **Реализация**: Сервер хранит `CurrentSecret` и `PreviousSecret`.
    - **Валидация**: Пробуем проверить текущим, если не вышло — старым.
    - **Деплой**: Позволяет обновить ключ в конфиге без разлогина всех пользователей сразу.
    - Проверяем старым пока не истекут все время жизни у старых токенов
19. **Расскажите о конкурентности в Go: чем отличаются goroutine, channel, mutex, RWMutex, atomic. Какие типичные race conditions встречаются в веб-бэкенде?**
    - **Goroutine**: Легкий поток под управлением рантайма Go.
    - **Channel**: Синхронизация через передачу данных («не общайтесь через память, разделяйте память через общение»).
    - **Mutex**: Блокировка доступа (один зашел — остальные ждут).
    - **RWMutex**: Много читателей, но только один писатель.
    - **atomic** — низкоуровневые операции на уровне CPU (CAS, добавление, загрузка), работают быстрее mutex, но применимы только для простых числовых операций и указателей.
    - **Race condition**: Часто в вебе — одновременная запись в мапу (map) или инкремент счетчика без мьютекса.
20. **Как вы проектируете bounded queue и backpressure для фоновых задач (worker pool)? Как вы решаете, когда возвращать 503, и какие альтернативы возможны?**

- **Дизайн**: Канал с буфером (`chan Task`) + Worker Pool (несколько горутин).
- **503**: Если канал полон, сразу возвращаем клиенту 503 Service Unavailable.

```
select {
case tasks<-newTask:
    // Либо добавлемя задачу в канал
default:
    // Либо тут отдаём клиенту 503 ошибку
}
```

``

- **Альтернативы**: Увеличение лимита или отклонение части запросов (Rate Limiting).

21. **Опишите graceful shutdown для фоновых воркеров: как остановить приём новых задач, как "осушить" очередь, как ограничить ожидание таймаутом, и какие есть подводные камни.**
    1. Закрываем канал с задачами (`close(tasks)`).
    2. Воркеры дочитывают то, что осталось в буфере, и завершаются.
    3. Ждем через `sync.WaitGroup` .
    4. Ограничиваем ожидание таймаутом (через `context`).

    > *Подвох:* Бесконечные задачи, которые не проверяют `ctx.Done()` .

22. **Объясните роль context.Context в веб-сервисе: откуда он берётся, как распространяется, как влияет на отмену запросов при disconnect клиента.**
    - **Роль**: Передача сигналов отмены, дедлайнов и метаданных.
    - **Источник**: В http рождается из `r.Context()` (контекста запроса).
    - **Отмена**: Если клиент закрыл соединение, контекст отменяется, и все вложенные вызовы (в БД, внешние API) должны тоже прекратиться, чтобы не тратить ресурсы.
23. **Как вы реализуете таймауты на уровне запроса и на уровне отдельных операций (DB, внешние вызовы)? Как вы выбираете значения таймаутов?**
    - Таймауты на уровне запроса реализуются через context.WithTimeout, создавая контекст с заданным временем жизни (например, 5 секунд для HTTP-запроса)
    - Этот контекст передаётся во все функции, выполняющие операции, поддерживающие контекст (DB, HTTP-клиенты).
    - Выбор значений таймаутов зависит от требований к производительности и пользовательскому опыту:
      - для HTTP-запросов обычно 3–10 секунд,
      - для внутренних вызовов — 1–3 секунды.
      - Важно учитывать среднее время обработки и пиковые нагрузки, а также обеспечить запас для сетевых задержек и обработки данных.
      - Таймауты должны быть достаточно короткими, чтобы не блокировать ресурсы

слишком долго, но достаточно длинными, чтобы позволить завершить нормальную работу в большинстве случаев.

24. **Опишите паттерн "parallel subcalls" с общим контекстом: как запускать несколько операций параллельно, отменять остальные при первой ошибке и избегать утечек goroutine.**
    Паттерн "parallel subcalls" реализуется с использованием errgroup.WithContext. Этот пакет предоставляет группу горутин, где при возврате первой ошибки контекст отменяется, а остальные операции могут отследить отмену и завершиться.
25. **Что такое errgroup? Как бы вы реализовали минимальную версию на стандартной библиотеке и какие гарантии она должна давать?**
    - **Суть**: Умная обертка над `sync.WaitGroup`, которая умеет возвращать первую возникшую ошибку.
    - **Минимальная версия**: `WaitGroup` для ожидания + `sync.Once` для фиксации первой ошибки + `context.CancelFunc` для остановки группы.

```go
package errgroup

import (
    "context"
    "sync"
)

type Group struct {
    ctx    context.Context
    cancel context.CancelFunc
    wg     sync.WaitGroup
    errOnce sync.Onces
    err     error
}

func WithContext(ctx context.Context) (*Group, context.Context) {
    ctx, cancel := context.WithCancel(ctx)
    return &Group{ctx: ctx, cancel: cancel}, ctx
}

func (g *Group) Go(f func() error) {
    g.wg.Add(1)
    go func() {
        defer g.wg.Done()
        if err := f(); err != nil {
            g.errOnce.Do(func() {
                g.err = err
                if g.cancel != nil {
                    g.cancel()
                }
            })
```

```go
            }
        }()
    }

    func (g *Group) Wait() error {
        g.wg.Wait()
        if g.cancel != nil {
            g.cancel()
        }
        return g.err
    }
```

26. **Объясните основные концепции gRPC: что такое service, method types (unary/streaming), proto-контракт, и как вы организуете версионирование API (v1/v2).**
    gRPC — это современный фреймворк для удаленного вызова процедур, работающий поверх HTTP/2.
    - Основой является **proto-контракт** (файл `.proto`), который служит «единым источником истины» для клиента и сервера.
    - В нем описываются сообщения (`message`) и **service** — набор методов, которые сервер обязуется реализовать.
    - gRPC поддерживает четыре типа методов:
      - **Unary** (классический запрос-ответ),
      - **Server Streaming** (один запрос — поток ответов),
      - **Client Streaming** (поток запросов — один ответ)
      - **Bidirectional Streaming** (полнодуплексный обмен данными).
        Версионирование API в gRPC обычно реализуется на уровне пакетов Protobuf.

        > Например, вы создаете директории `proto/v1` и `proto/v2`. В самом файле указывается `package api.v1;`. Это позволяет генерировать разные структуры в Go и запускать два варианта сервиса одновременно на одном сервере, пока клиенты плавно переходят на новую версию. Главное правило — не ломать обратную совместимость (не менять номера полей и не удалять их)

27. **Как устроены interceptors в gRPC и чем они похожи на HTTP middleware? Какие задачи вы бы вынесли в interceptors (логирование, ..., ...) и почему?**
    Interceptors (перехватчики) — это прямой аналог HTTP middleware в мире gRPC.
    Существуют два типа перехватчиков:
    - **Unary** (для обычных запросов)
    - **Stream** (для потоков).
      Как и в случае с middleware, их можно собирать в цепочки (chaining), где каждый

следующий перехватчик вызывает обертку над основным хендлером.
В interceptors обычно выносят:

- логирование (запись вызова, аргументов, длительности),
- аутентификацию (извлечение и проверка токена из метаданных)
- трассировку (проброс request_id, OpenTelemetry)
- метрики (счётчики, гистограммы)
- rate limiting, retry/timeout, валидацию входных сообщений

28. **Как вы пишете тесты в Go для HTTP: чем отличаются unit и integration тесты, как использовать httptest?**
    - **Unit-тесты** в вебе проверяют логику конкретного хендлера в изоляции. Для этого все внешние зависимости (база, API) заменяются моками (mock).
    - **Integration-тесты** проверяют связку: роутинг + middleware + хендлер + реальная (или тестовая в Docker) база данных. Основной инструмент здесь — пакет `net/http/httptest`.
      Для юнит-тестов мы используем `httptest.NewRecorder()`, который имитирует `ResponseWriter` и «записывает» ответ сервера в память. Это позволяет проверить статус-код и тело ответа без запуска реального сервера. Для интеграционных тестов можно использовать `httptest.NewServer()`, который поднимает сервер на случайном свободном порту, позволяя прогнать через него честные HTTP-запросы с помощью `http.DefaultClient`.
29. **Как вы пишете тесты в Go для gRPC, как тестировать gRPC через bufconn?**
    Тестирование gRPC-сервисов часто упирается в необходимость поднимать сервер на TCP-порту, что может привести к конфликтам при параллельном запуске тестов. Решение — использование пакета `google.golang.org/grpc/test/bufconn`. Он позволяет создать слушатель (`net.Listener`), который работает полностью в оперативной памяти (in-memory).

    Вы создаете буферное соединение, передаете его в `grpc.Serve()` и настраиваете клиент так, чтобы он использовал специальный `Dialer`, подключающийся к этому буферу, а не к сети. Это делает тесты:
    1. **Быстрыми**: Нет накладных расходов на сетевой стек ОС.
    2. **Изолированными**: Вам не нужно беспокоиться о занятых портах или настройках брандмауэра.
    3. **Детерминированными**: Тесты не упадут из-за того, что порт `50051` внезапно оказался занят другой программой.
30. **Опишите стратегию наблюдаемости: structured logging через log/slog (attrs, groups, levels), correlation/request_id (HTTP headers и gRPC metadata), метрики и базовые идеи трассировки (spans, propagation). Как вы бы связали логи, метрики и трейсы для диагностики инцидентов?**
    Логи, Метрики и Трейсы.

    - **Structured Logging (`log/slog`)**: Запись логов в JSON с атрибутами (`user_id`, `request_id`). Это позволяет не «грепать» текст, а делать точные запросы по

полям.

- **Metrics**: Числовые показатели (количество 500-х ошибок, время выполнения запроса). Помогают понять, *что* система чувствует себя плохо прямо сейчас.
- **Tracing**: Визуализация пути запроса через все микросервисы в виде спанов (spans). Помогает понять, *где именно* произошла задержка или ошибка.

Как это связать? При входе запроса генерируется **Correlation ID (Trace ID)**. Он передается в заголовках HTTP или Metadata gRPC. Этот ID записывается в каждый лог и прикрепляется к каждому трейсу. **Диагностика инцидента:** Вы видите на графике в Grafana всплеск ошибок (Метрики). Кликаете на подозрительный интервал, переходите в Трейсинг и видите, что запрос «застрял» в сервисе заказов. Копируете Trace ID, вставляете в систему сбора логов и видите детальные ошибки из кода именно этого проблемного запроса.

by Volkov Roman